Файл `.gitlab-ci.yml`. Best practices. Helm-чарты

Занятие 10 · День 5, утро

План

1. Анатомия `.gitlab-ci.yml`: stages, jobs, image, script
2. Переменные: встроенные, свои, защищённые, файлы
3. Когда запускать: `rules`, `workflow`, `when`
4. Ускорение: `needs`, `cache`, `artifacts`, `interruptible`
5. Переиспользование: `extends`, `!reference`, `include`, CI/CD components
6. Сборка образа и push в Container Registry
7. **Helm**: свой чарт для приложения
8. Best practices

Анатомия

```yaml
stages: [test, build, deploy]       # порядок этапов

variables:                          # глобальные переменные
  IMAGE: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA

default:                            # значения по умолчанию для всех jobs
  image: alpine:3.22
  tags: [docker]

unit-tests:                         # имя job (любое, кроме зарезервированных)
  stage: test
  image: python:3.13-alpine         # переопределяем default
  before_script:
    - python --version
  script:                           # обязательная часть: команды shell
    - python -m unittest -v
  after_script:
    - echo "выполнится даже при ошибке"
```

Любая команда `script` вернула ≠ 0 → job **failed** → следующие stages не запускаются.

Переменные

Источник	Пример	Где задаются
Предопределённые	`CI_COMMIT_SHORT_SHA`, `CI_COMMIT_BRANCH`, `CI_COMMIT_TAG`, `CI_PIPELINE_SOURCE`, `CI_REGISTRY_IMAGE`, `CI_DEFAULT_BRANCH`	GitLab сам
В файле	`variables:` глобально или в job	`.gitlab-ci.yml` (не для секретов!)
В настройках	Settings → CI/CD → Variables	проект / группа / инстанс

Флаги переменных в настройках:

- **Masked** — скрыта в логах (`[MASKED]`)
- **Protected** — только для защищённых веток и тегов
- **Type: File** — содержимое пишется во временный файл, в переменной — **путь** (идеально для kubeconfig)
- **Environment scope** — разное значение для `staging` / `production`

`rules` — когда запускать job

```yaml
build-image:
  script: docker build ...
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH     # main → да
    - if: $CI_COMMIT_TAG                               # тег → да
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
      changes: [Dockerfile, app.py]                    # MR → только если менялись файлы
      when: manual                                     # …и по кнопке
    # ни одно правило не подошло → job не добавляется
```

- Правила проверяются **по порядку**, срабатывает первое подходящее
- `when`: `on_success` (по умолчанию), `manual`, `delayed`, `always`, `never`
- `only` / `except` — **устаревший** синтаксис, используйте `rules`

`workflow` — когда создавать пайплайн

```yaml
workflow:
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH && $CI_OPEN_MERGE_REQUESTS
      when: never                  # есть открытый MR → не дублируем branch-пайплайн
    - if: $CI_COMMIT_BRANCH
    - if: $CI_COMMIT_TAG
```

Без этого push в ветку с открытым MR может создать **два** пайплайна.

`needs` — граф вместо этапов

```yaml
lint:        { stage: test,   script: [make lint] }
unit-tests:  { stage: test,   script: [make test] }
build-image:
  stage: build
  needs: [unit-tests]          # стартует, как только прошли тесты, не ждёт lint
  script: [docker build ...]
deploy:
  stage: deploy
  needs: [build-image]
```

- **DAG** — пайплайн быстрее: job ждёт только то, от чего зависит
- `needs: []` — стартовать сразу, независимо от этапов

Cache и artifacts — не путать

	cache	artifacts
Зачем	ускорить: зависимости (`.venv` , `node_modules` , `.m2`)	передать **результат**: бинарник, отчёт, манифест
Гарантия	**нет**, best effort	**да**
Между	пайплайнами	jobs одного пайплайна (+ скачать из UI)
Срок	пока не вытеснен	`expire_in`

```yaml
cache:
  key:
    files: [requirements.txt]      # новый ключ при изменении зависимостей
  paths: [.cache/pip]
artifacts:
  paths: [dist/]
  reports:
    junit: report.xml              # результаты тестов прямо в MR
  expire_in: 1 week
```

Не повторяемся: `extends`, `!reference`, `include`

```yaml
.deploy:                          # скрытый job-шаблон (точка в начале)
  image: alpine/k8s:1.36.4
  script:
    - helm upgrade --install shop-api chart/ -n "$NAMESPACE" ...

deploy-staging:
  extends: .deploy
  variables: { NAMESPACE: shop-staging }

deploy-prod:
  extends: .deploy
  variables: { NAMESPACE: shop-prod }
  before_script:
    - !reference [.deploy, script]   # взять кусок из другого job
```

`include` — пайплайн из частей

```yaml
include:
  - local: ci/lint.yml
  - project: platform/ci-templates
    ref: v2.1.0
    file: /docker-build.yml
  - component: $CI_SERVER_FQDN/components/sast/sast@1.2.0   # CI/CD component
```

CI/CD Components и каталог

- **Component** — переиспользуемый кусок пайплайна с **входными параметрами** (`spec:inputs`) и **версией**
- Публикуются в **CI/CD Catalog** GitLab — как библиотека пайплайнов для всей компании
- Подключение по версии: `component: …/docker-build@1.4.0`

```yaml
# templates/docker-build.yml в проекте-компоненте
spec:
  inputs:
    stage:      { default: build }
    dockerfile: { default: Dockerfile }
---
docker-build:
  stage: $[[ inputs.stage ]]
  script: docker build -f $[[ inputs.dockerfile ]] -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA .
```

Платформенная команда пишет компоненты — продуктовые команды подключают одной строкой.

Полезные ключевые слова

Ключ	Зачем
`interruptible: true`	новый push отменяет устаревший пайплайн
`resource_group: prod`	деплои в одно окружение — строго по очереди
`retry: { max: 2, when: [runner_system_failure] }`	повтор при сбое инфраструктуры
`timeout: 10m`	не висеть часами
`allow_failure: true`	job может упасть, пайплайн — нет (линтеры в процессе внедрения)
`parallel: { matrix: [...] }`	один job на набор параметров
`environment: { name: production, url: … }`	история деплоев, кнопка отката
`services: [postgres:17]`	сопутствующий контейнер на время job

ДЕМО Сборка и публикация образа

```yaml
build-image:
  stage: build
  image: docker:29.9
  before_script:
    - echo "$CI_REGISTRY_PASSWORD" | docker login -u "$CI_REGISTRY_USER" --password-stdin "$CI_REGISTRY"
  script:
    - docker build --build-arg APP_VERSION="$CI_COMMIT_SHORT_SHA" -t "$IMAGE" .
    - docker push "$IMAGE"
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
    - if: $CI_COMMIT_TAG
```

- `CI_REGISTRY_USER` / `CI_REGISTRY_PASSWORD` — временные, на время job
- Тег = **SHA коммита** — однозначно связывает образ и код; `latest` — не для деплоя
- Docker берётся с хоста runner'a через `docker.sock`

Best practices `.gitlab-ci.yml`

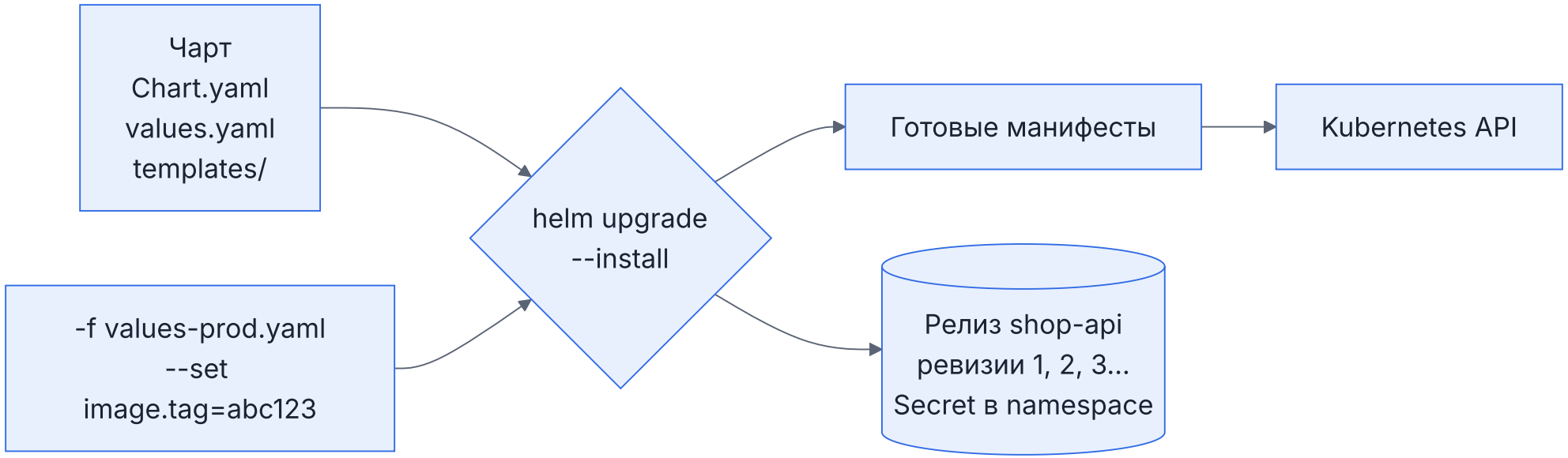
- **Быстрая обратная связь**: дешёвые проверки (lint, unit) первыми, `needs`, `interruptible`
- **Фиксируйте версии** образов (`python:3.13-alpine`, а не `python:latest`) и `include` (`ref:`)
- **Секреты — только в переменных** (masked + protected), никогда в файле и логах
- **Собрали один раз — деплоим везде**: один образ проходит staging → production
- `rules` **вместо** `only/except`, `workflow` против дублей
- **DRY**: `extends`, `include`, компоненты; но без «магии» на 5 уровней
- **Production — вручную или по тегу**, `resource_group`, protected runner
- **Проверяйте** CI-файл: Build → **Pipeline editor** (валидация, визуализация, merged YAML)

Helm-чарты

Упаковываем приложение для деплоя из CI

Зачем свой чарт

- Манифесты почти одинаковые для staging и production — отличаются **значения**: тег образа, реплики, ресурсы, домен
- Один `helm upgrade --install` вместо пачки `kubectl apply`
- **Релиз с историей**: `helm history`, `helm rollback`
- Удаление — одной командой, без «забытых» объектов



Структура чарта

```
helm create shop-api        # генерирует пример — удобно как шпаргалка
```

```
chart/
├── Chart.yaml              # имя, версия чарта, appVersion
├── values.yaml             # параметры по умолчанию
└── templates/
    ├── _helpers.tpl        # именованные шаблоны (метки, имена)
    ├── configmap.yaml
    ├── deployment.yaml
    ├── service.yaml
    └── NOTES.txt           # подсказка после установки
```

```yaml
# Chart.yaml
apiVersion: v2
name: shop-api
version: 0.1.0        # версия ЧАРТА — меняем при изменении шаблонов
appVersion: "1.0.0"   # версия ПРИЛОЖЕНИЯ
```

Шаблоны: Go templates

```yaml
# values.yaml
replicaCount: 2
image:
  repository: registry.192.168.56.200.nip.io/demo/shop-api
  tag: ""
service: { type: LoadBalancer, port: 80 }
```

```yaml
# templates/deployment.yaml (фрагмент)
metadata:
  name: {{ include "shop-api.fullname" . }}
  labels:
    {{- include "shop-api.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      {{- with .Values.imagePullSecrets }}
      imagePullSecrets:
        {{- toYaml . | nindent 8 }}
      {{- end }}
      containers:
        - image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion }}"
```

`.Values`, `.Release.Name`, `.Release.Namespace`, `.Chart.*` · функции: `default`, `quote`, `toYaml`, `nindent`, `sha256sum` · `{{-` убирает пробелы слева

Команды Helm для разработки чарта

```
helm lint chart/                                  # проверка
helm template shop-api chart/ | less              # отрендерить локально, без кластера
helm template shop-api chart/ --set replicaCount=3 -f values-prod.yaml

helm upgrade --install shop-api chart/ -n shop --create-namespace \
  --set image.tag=1a2b3c4 --wait --timeout 3m
helm list -n shop
helm history shop-api -n shop
helm get values shop-api -n shop                  # с какими значениями установлен
helm get manifest shop-api -n shop
helm rollback shop-api 1 -n shop
helm uninstall shop-api -n shop
```

Приоритет значений: `values.yaml` < `-f file` < `--set` (последний выигрывает).

Хитрости чартов

- **Перезапуск при смене ConfigMap**:
  `checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}`
- `--wait` — ждать готовности; в Helm 4 используется **kstatus**
- `--rollback-on-failure` (Helm 4; в Helm 3 — `--atomic`) — откатить, если не дождались
- `values.schema.json` — валидация values, понятные ошибки
- Хранение чартов: **OCI-registry** (`helm push chart.tgz oci://…`) — в том числе GitLab
- Зависимости (`dependencies:` в Chart.yaml): PostgreSQL, Redis и т.д. из других чартов

ДЕМО Демо: chart + пайплайн

```
cd ~/shop-api
mkdir -p chart/templates
helm lint chart/ && helm template shop-api chart/ | head -40

# ручная проверка в кластере (образ уже собран CI)
helm upgrade --install shop-api chart/ -n shop-test --create-namespace \
  --set image.tag=$(git rev-parse --short HEAD) --wait
kubectl -n shop-test get pods,svc      # ImagePullBackOff? — registry приватный!
```

В пайплайн: job `lint-chart` (stage `test`) — `helm lint` + `helm template`.

Почему образ не скачался и как дать кластеру доступ к registry — **следующее занятие**.

Итоги

- `.gitlab-ci.yml`: **stages → jobs → script**, `default`, `variables`
- `rules` + `workflow` — когда; `needs`, `cache`, `artifacts`, `interruptible` — быстро
- `extends` / `include` / **components** — без копипасты
- Секреты — в переменных (**masked, protected, File**)
- Образ: тег = SHA, собрать один раз — деплоить везде
- **Helm-чарт**: шаблоны + values; `lint`, `template`, `upgrade --install`, `rollback`

Практика 10

Пайплайн и Helm-чарт для notes-api

Задание — на платформе testlms.kz